

Problems that can solve using Dynamic prog.

- ① longest common subsequence
- ② matrix chain multiplication
- ③ 0/1 knapsack Problem
- ④ multistage Graph
- ⑤ optimal Binary search tree
- ⑥ Travelling salesman person Problem
- ⑦ All pair shortest path problem
- ⑧ Sum of subset Problem
- ⑨ Bellman Ford Algo
- ⑩ Reliability Design problem

Ex $\Rightarrow$

	W	P
I_1	2	1
I_2	3	2
I_3	4	5

find max profit using
o/a knapsack?

knapsack = 6

Soln $\Rightarrow$

	I_3	I_1
Weight	$\downarrow$	$\downarrow$
	4	2
	$\downarrow$	$\downarrow$
	5	1

maximum

6 Profit

$\rightarrow I_2$ is left out from knapsack

using subproblem approach

Pi	Wi	0	1	2	3	4	5	6
1	2	0	0	1	1	1	1	1
2	3	0	0	1	2	2	3	3
5	4	0	0	1	2	5	5	6

Optimal Result
Dynamic Prog.

Greedy Approach
(Feasible Result, not guaranteed optimal result.)

classmate
Date
Page

Fractional Not allowed here.

① 0/1 knapsack problem $\Rightarrow$ Either complete object or No object included

Objective is maximize

$$\sum_{i=1}^n x_i^o \times p_i^o$$

Subject to constraint,

$$\sum_{i=1}^n (x_i^o \times w_i^o) \leq m$$

x_i^o is Either 0 or 1

p_i^o = profit of object

w_i^o = weight of object

x_i^o = portion of object

$$\text{knapsack}(n, w) = \max \left(\text{knapsack}(n-1, w - w_n) + p_n, \text{knapsack}(n-1, w) \right)$$

Time Complexity = $O(Nm)$

N = num(objects)

m = (knapsack capacity)

Ex $\Rightarrow$

	Weights	Values
I_1	10	60
I_2	7	28
I_3	4	20
I_4	2	24

Knapsack = 11 kg

V_{opt} = The total value of items picked up by an optimal algorithm

V_{greedy} = The total value of items picked by the greedy algo.

Find the value of $(V_{opt} - V_{greedy}) = ?$

Soln $\Rightarrow V_{opt} = I_1 \rightarrow 10 \Rightarrow 60$ profit { highest using 0/1 knapsack

$$V_{greedy} \Rightarrow I_1 = \frac{60}{10} = 6$$

$$I_2 = \frac{28}{7} = 4$$

$$I_3 = \frac{20}{4} = 5$$

~~But~~

$$I_4 = \frac{24}{2} = 12$$

max value's item

I_1 Not possible w 10, Next max value

$$\therefore V_{greedy} = 28 + 24 + 20$$

$$= 72$$

$I_2 = 7$

$I_3 = 20$

$I_4 = 24$

Max Profit item

Knapsack = 11

$\therefore$ value $(V_{opt} - V_{greedy})$

$$= 60 - 44 = 16$$

Date _____
Page _____

	Profit	Weight
<u>Ex</u> $\Rightarrow$ I_1	1	1
I_2	4	3
I_3	5	4
I_4	7	5

knap sack = 7

By using 0/1 knapsack
find max profit?

Solⁿ

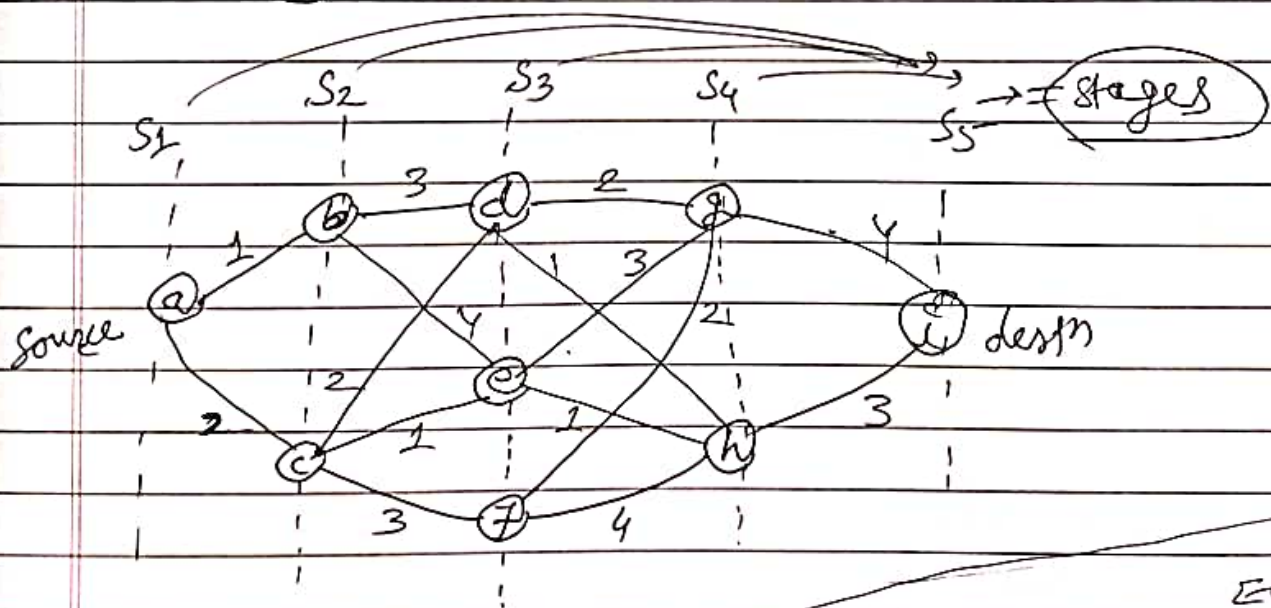
$$\begin{array}{ccc}
 & I_2 + I_3 & \\
 & \downarrow \quad \downarrow & \\
 W = & 3 \quad 4 & \\
 & \downarrow \quad \downarrow & \\
 P = & 4 \quad 5 & = 9 \\
 & & \underline{\underline{=}}
 \end{array}$$

multistage graph

- Find shortest path from source to destn.
- Assume all Edges direction are in forward direction.
- Divide the graph ~~in~~ stages
- It uses 2 method for cost computing

① Forward

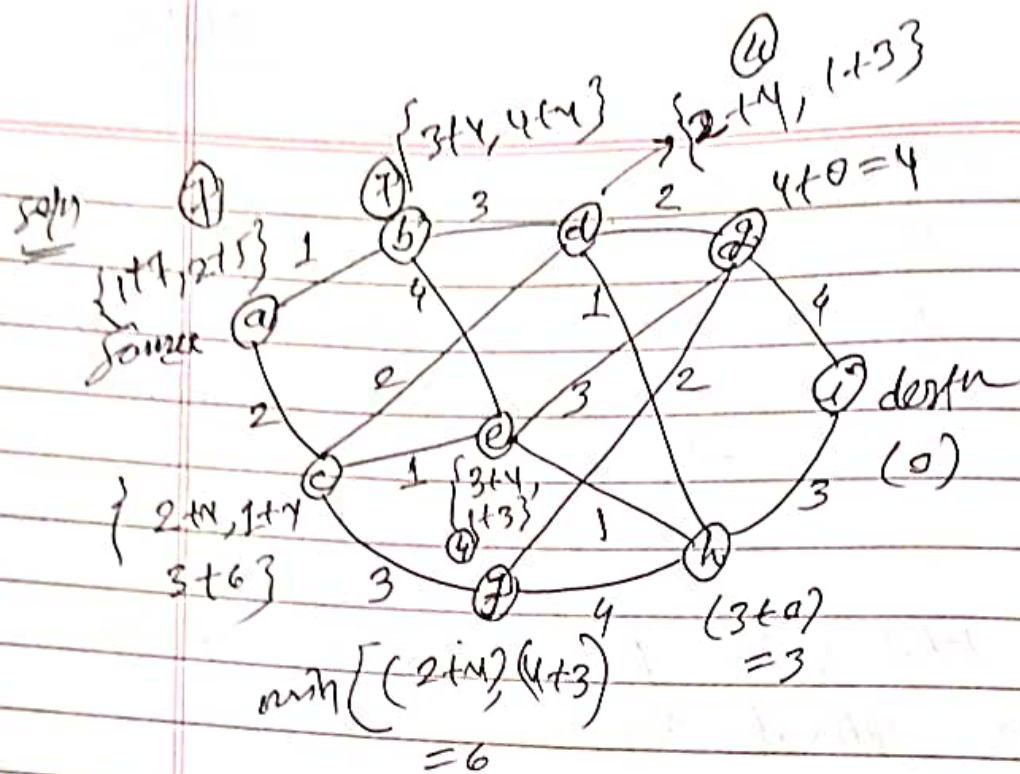
② Backward



$$\text{cost}(i, j) = \min \{ \text{cost}(i+1, l) + c[l, j] \}$$

↑ i stage
 ↑ j node of stage i
 ↑ $c[l, j]$ Edge cost

$$\text{cost}(4, g) = \{ \text{cost}(5, i) + c(l, g) \}$$



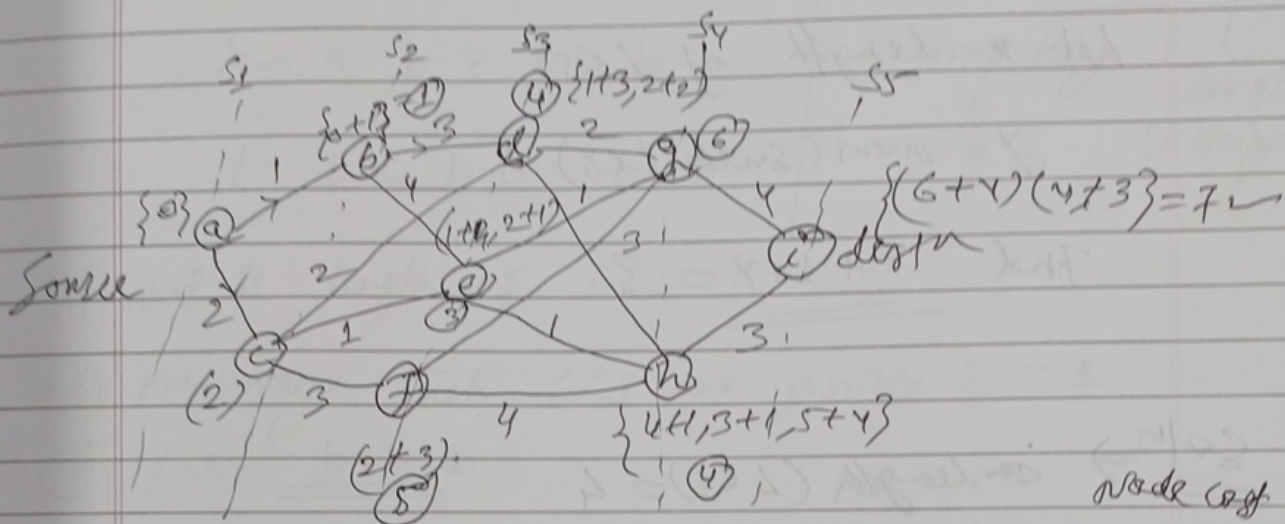
forward approach $\rightarrow$ start from destn
 $\therefore$ Always has path of forward direction
 shortest path = a, c, e, h, i
 cost = 7

multisource graph

complexity = $O(V+E)$

Backward Approach

- make arrows towards forward direction
- start from source



$$bcost(s, i) = \min \left\{ c(i, l) + \overbrace{bcost(i-1, l)}^{\text{node cost}} \right\}$$

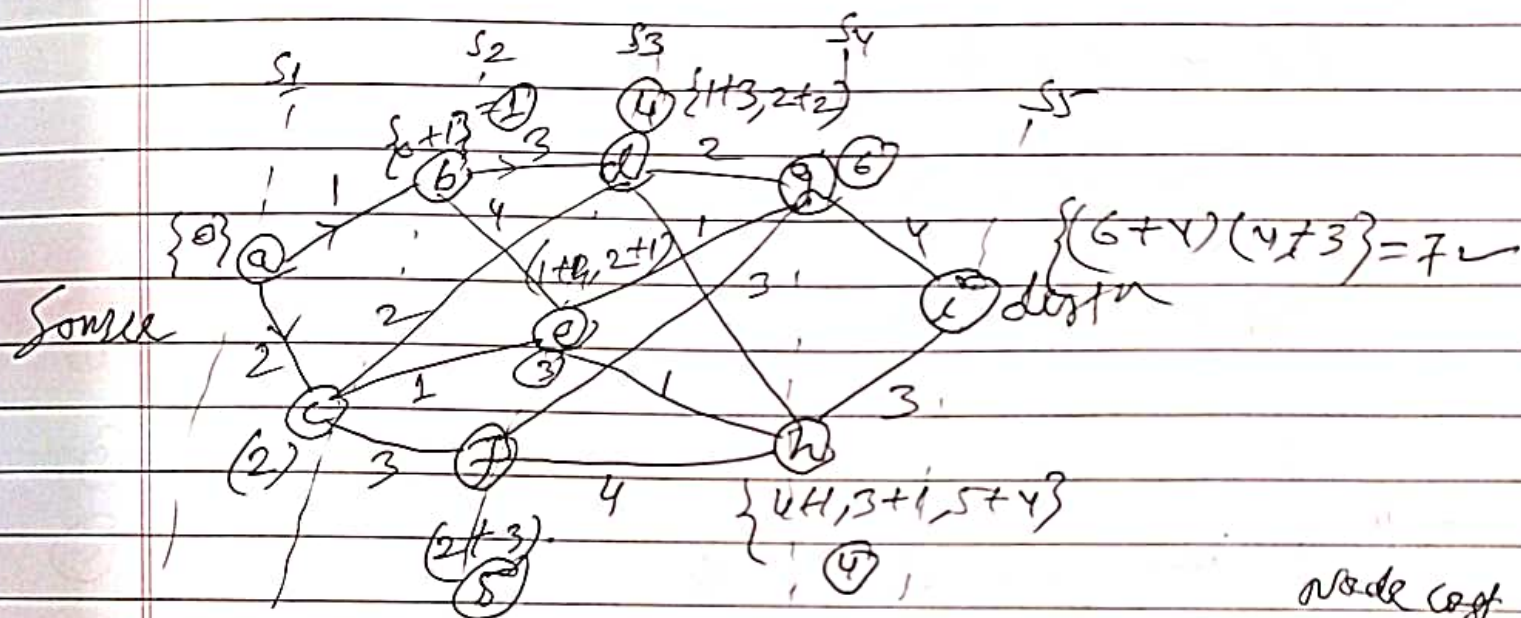
stage
node of stage i
prev stage
node of stage (i-1)

path a to i $\Rightarrow$ a, c, e, h, i ?
 cost = 7
 same path & cost

$$\text{complexity} = O(V+E)$$

Backward Approach

- make arrows towards forward direction
- start from source



$$bcost(s, i) = \min \left\{ c(i, l) + \overbrace{bcost(i-1, l)}^{\text{node cost}} \right\}$$

Annotations for the equation above:

- s : stage
- i : node of stage i
- $i-1$: per stage
- l : node of stage $(i-1)$

path a to $i \Rightarrow a, c, e, h, i$?
 $\text{cost} = 7$
 same path & cost

$$\text{complexity} = O(V + E)$$

multistage graph summary

- ① Graph is divided into multiple stage.
 - ② Same stage vertex doesn't connected to each other.
 - ③ Forward approach
 - ④ Backward Approach
- } $O(V+E)$
- ⑤ Dynamic programming Approach

"Longest Common Subsequences" (LCS)

→ Dynamic prog Approach

$X = a \ b \ c \ b \ c \ b \ a$

$Y = b \ c \ b \ b \ c \ b \ b$

Complexity = $O(m \times n)$

length of string
 x

length
of string
 y

① what is length of LCS ? = 5

② How many such LCS are possible ? = 1

③ LCS = bcbcb

$Y_i \backslash X_j$ 0 a b c b c b a

0 0 0 0 0 0 0 0

b 0 0 1 1 1 1 1

c 0 0 1 2 2 2 2

b 0 0 1 1 2 3 3

b 0 0 1 2 3 3 4

c 0 0 1 2 3 4 4

b 0 0 1 2 3 4 5

b 0 0 1 2 3 4 5

diagonal show same LCS

longest
common
subsequence

$C[i, j] = 0$. iff $i = 0$ or $j = 0$

iff $x_i \neq y_j$ then $C[i, j] = \max(C[i-1, j], C[i, j-1])$

iff $x_i = y_j$, $C[i, j] = C[i-1, j-1] + 1$

Ex $\Rightarrow A = 21222$

$B = 1212221$

Let $x = \text{length of LCS}$

$y = \text{num (such LCS)}$

Find $x + 10y = ?$

Soln $\Rightarrow$ length (LCS) = 4

num(LCS) =	① 1222	④
	② 2122	
	③ 2122	

$\therefore 4 + 3 \times 10 = 34$

* Matrix chain multiplication

$$A [p \times q] \quad B [q \times r] \quad C [r \times s]$$

$$p=5, q=4, r=6, s=2$$

$$① \quad A \times B \neq B \times A \quad \left\{ \begin{array}{l} \text{matrix} \\ \text{Not commutative always} \end{array} \right.$$

$$② \quad (A \times B) \times C = A \times (B \times C) \quad \left\{ \begin{array}{l} \text{matrix} \\ \text{Associative} \end{array} \right.$$

multiply different way = 2 for

Follow order
of A, B, C

Formula, if $A_1, A_2, A_3, \dots, A_n$

$$A_1 \times A_2 \times A_3 \times \dots \times A_n$$

Then num (ways) to multiply is,

$$\frac{[2(n-1)]!}{n! \times (n-1)!} \quad \left. \vphantom{\frac{[2(n-1)]!}{n! \times (n-1)!}} \right\} \text{ gives same Result.}$$

Ex → For A_1, A_2, A_3, A_4 find num (way) to multiply,

$$\underline{\text{soln}} = \frac{[2(4-1)]!}{4! \times (4-1)!} = \frac{6!}{4! \times 3!} = \frac{6 \times 5}{3 \times 2} = \underline{\underline{5 \text{ ways}}}$$

Soln

$$(A \times B) \times C = A \times (B \times C)$$

$$\left[\begin{array}{cc} A_{5 \times 4} & \times B_{4 \times 6} \end{array} \right] \times C_{6 \times 2}$$

5×6

$$= 5 \times 4 \times 6 + 5 \times 6 \times 2$$

$$= 120 + 60 = \underline{\underline{180}}$$

$$A_{5 \times 4} \times \left[\begin{array}{cc} B_{4 \times 6} & \times C_{6 \times 2} \end{array} \right]$$

$$5 \times 4 \times 2 + 4 \times 6 \times 2$$

$$40 + 48$$

$$\underline{\underline{88}}$$

Dynamic programming for matrix chain multiply

(optimal) $\hookrightarrow$ min num (mult/div) = Result Select min

Ex $\Rightarrow A_1 [P_0 \times P_1], A_2 [P_1 \times P_2], A_3 [P_2 \times P_3],$
 $A_4 [P_3 \times P_4]$

$P_0 = 5, P_1 = 4, P_2 = 6, P_3 = 2, P_4 = 7$

$m[i, j] = \begin{cases} 0 & \text{if } i=j \text{ or } i > j \\ \min_{i \leq k < j} [m[i, k] + m[k+1, j]] + \end{cases}$

order of multiplication

$A_1 \text{ to } A_4 \Rightarrow (A_1 \text{ to } A_3) \times A_4$

$\Rightarrow A_1 \times (A_2 \times A_3) \times A_4$

154

	1	2	3	4
4	154	104	84	0
3	88	48	0	0
2	120	0	0	0
1	0	0	0	0

$m[i, j] \Rightarrow$

Time Complexity

$n \times n \times n = n^3$

$O(n^3)$

for loop

$m[1, 2] \Rightarrow$
 $k=1 [m[1, 1] + m[2, 2] + P_0 P_1 P_2]$

$= 0 + 0 + 4 \times 5 \times 6$

$= 120$

$m[2, 3] \Rightarrow$
 $k=2 [m[2, 2] + m[3, 3] + P_1 P_2 P_3]$
 $0 + 0 + 4 \times 6 \times 2 = 48$

244

= 120

$$0 + 104 + 5 \times 4 \times 7$$

$$k=1 \quad m[1,1] + m[2,4] + P_0 P_1 P_4$$

$$m[1,4] \quad k=2 \quad m[1,2] + m[3,4] + P_0 P_2 P_4$$

$$k=3 \quad 0 + 84 + 5 \times 6 \times 7 = 294$$

min value = 154

$$m[1,3] + m[4,4] + P_0 P_3 P_4$$

$$84 + 0 + 5 \times 2 \times 7 = 154$$

$$84 + 84 = 168$$

$$m[2,4] \quad k=2 \quad m[2,2] + m[3,4] + P_1 P_2 P_4$$

$$0 + 0 + 4 \times 6 \times 7 = 168$$

$$k=3$$

$$252$$

select min

(104)

$$m[2,3] + m[4,4] + P_1 P_3 P_4$$

$$48 + 0 + 4 \times 2 \times 7 = 104$$

$$m[3,4] \Rightarrow$$

$$k=3, (m[3,3] + m(3,4)$$

$$+ P_2 P_3 P_4)$$

$$= 0 + 0 + 6 \times 2 \times 7$$

$$= 84$$

$$m[1,3] \Rightarrow$$

$$m[1,1] + m[2,3] + P_0 P_1 P_3$$

$$k=1 \text{ or } k=2$$

$$0 + 48 + 5 \times 4 \times 2 = 120$$

$$88$$

select min

of

$$k=1$$

$$\text{or } k=2$$

$$m[1,2] + m[3,3] + P_0 P_2 P_3$$

$$120 + 0 + 5 \times 6 \times 2$$

$$88$$

A Sum of Subset problem

↳ only dynamic Prog, Not DAC

$$\text{set} = \{a_1, a_2, a_3, \dots, a_n\}$$

$$\text{sum} = W$$

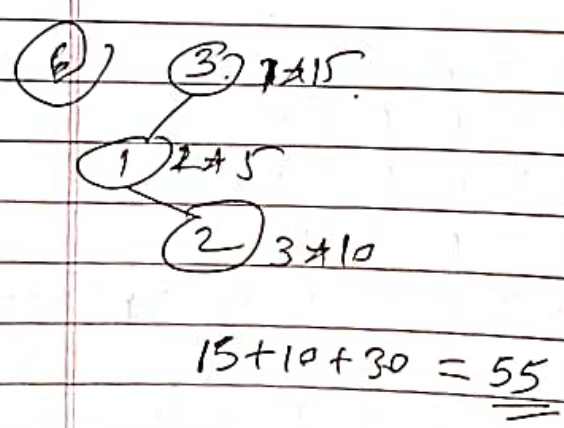
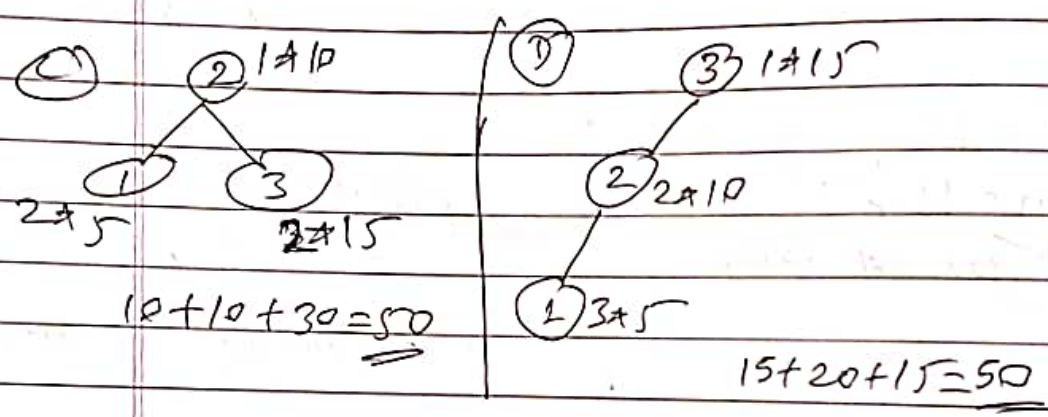
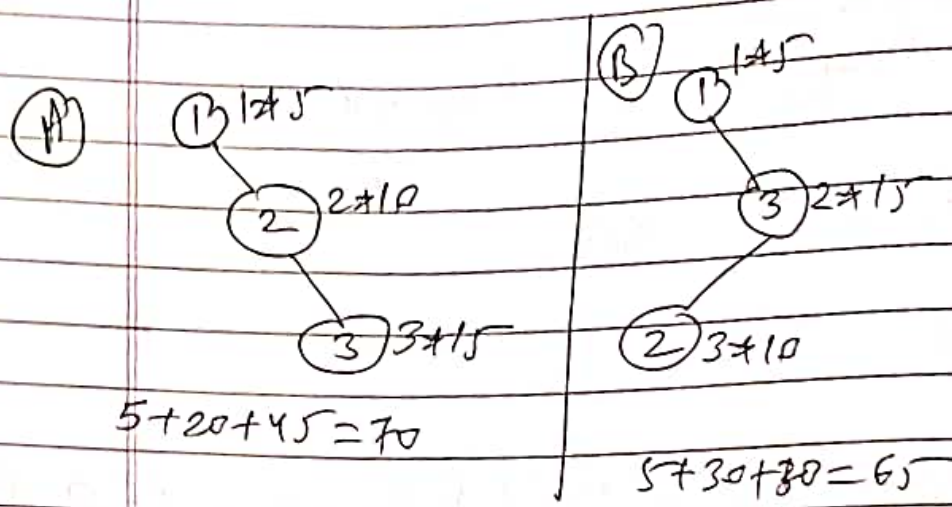
$$X[i, j] = X[i-1, j] \vee X[i-1, j-a_i] \text{ if } 2 \leq i \leq n \text{ \& } a_i \leq j \leq W$$

Ex → set = {2, 3, 7, 8, 10}
 sum = 11
 ✓ yes set exists.

Complexity = $O(n \times W)$
 n ← num(elements)
 W ← sum

0	0	1	2	3	4	5	6	7	8	9	10	11
0	T	F	F	F	F	F	F	F	F	F	F	F
2	T	F	T	F	F	F	F	F	F	F	F	F
3	T	F	T	T	F	T	F	F	F	F	F	F
7	T	F	T	T	F	T	F	T	F	T	T	F
8	T	F	T	T	F	T	F	T	T	T	T	T
10	T	F	T	T	F	T	F	T	T	T	T	T

{3, 8}



optimal BST = (C) & (D) = 2

Note = More than one optimal BST are possible.

① Complete Graph for n nodes
 $V = n$
 $E = \frac{n(n-1)}{2}$
 $\therefore \text{Complexity} = O(V^3)$
 $\therefore E = n^2 \therefore E = n^2 \rightarrow n \times n^2 = n^3$

Bellman ford Algo

→ shortest path algo

→ It gives correct result (shortest path) for (-)ve Edge.

→ It has capacity to find (-)ve cycle in the graph

② Relax $(V-1) + 1$ times
 $\therefore O(V \times E)$

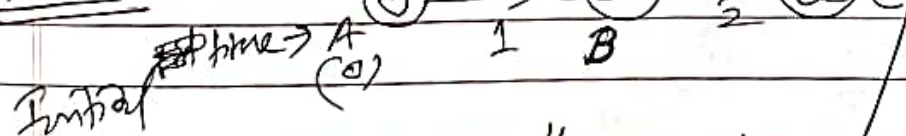
→ Relax() = if $(n-1)$ calling = shortest path

③ For cycle graph
 $V = E = n$
 $\therefore O(n^2)$

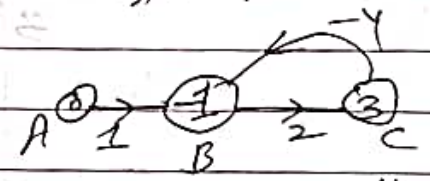
if n th calling = shortest path update
 $\therefore$ It contain (-)ve edge cycle

$\infty + \text{Anything} = \infty$
 $\infty - \text{Anything} = \infty$

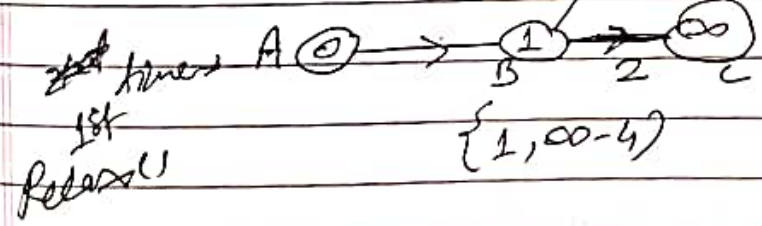
Relax()



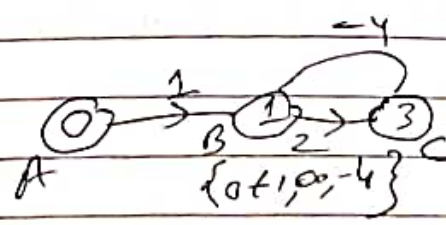
But call 3rd Relax() one more time,



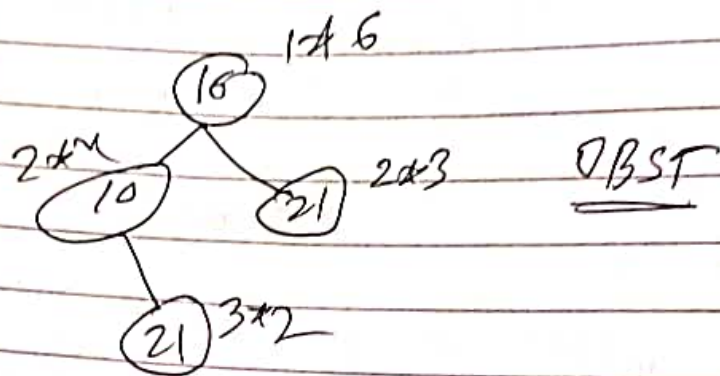
Shortest path or Graph Update (B)
 $\therefore$ Graph contain (-)ve edge cycle.



2nd time Relax()

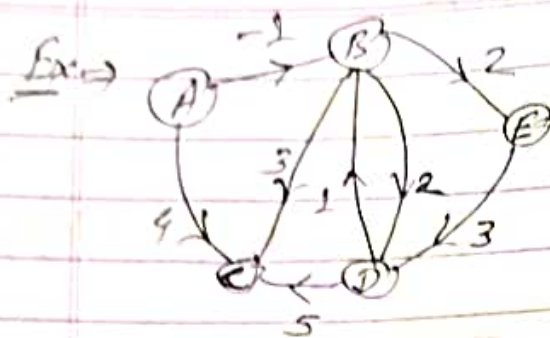


} $n = 3$
 $n - 1 = 2$ time we get shortest path



$$6 + 8 + 6 + 6 = 26 \text{ (3)}$$

Note → During Relaxing, if any Intermediate steps gives repeated (same) result then Graph's shortest path will confirmed found.
No need to check further.



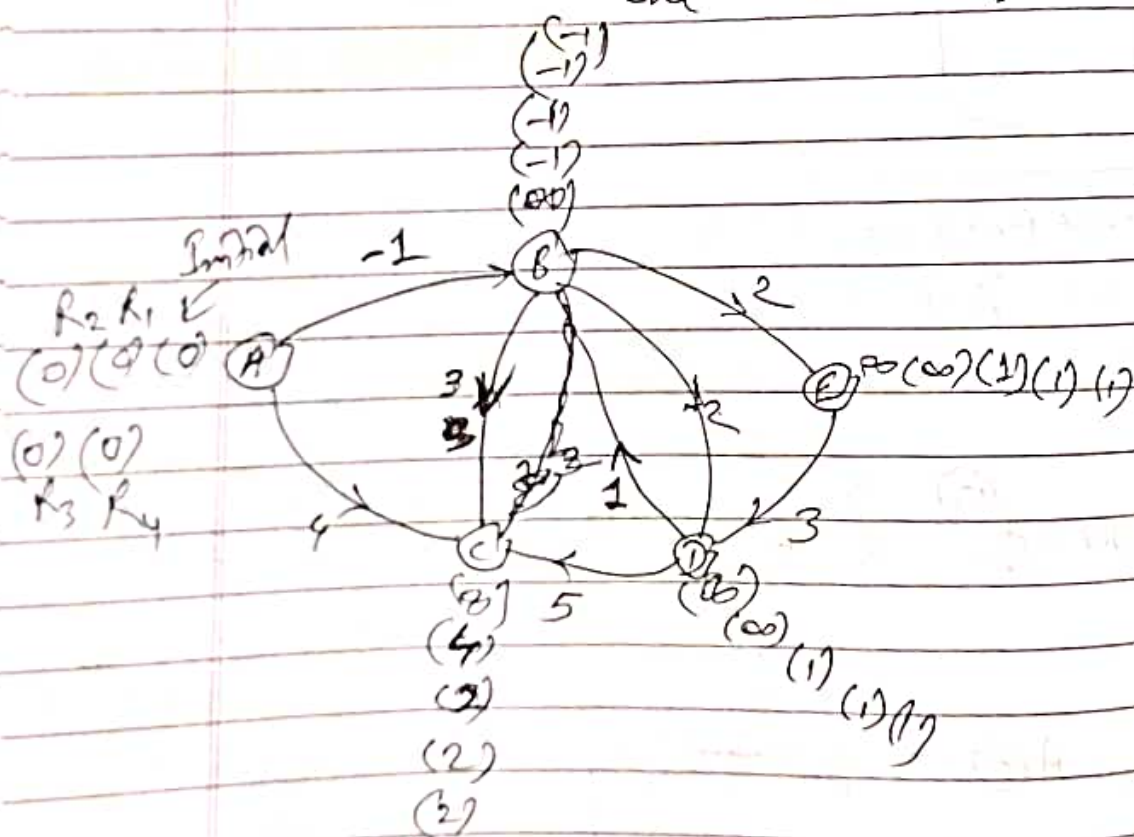
Apply Bellford
Algo to

check whether
we can find minimum
shortest path for this
graph or
not?

Soln →

vertices = 5

Total edges = num (lines Rel(A))
= 5-1 = 4



Relax = $R_2 \& R_4$ gives same result & we need
to go further

∴ By using Bellmanford Algo
we find shortest path of this algo

All Pair shortest path (Floyd Warshall Algo)

→ Dynamic Programming Approach

→ Find Each vertex's shortest path

→ Complexity = n^2 for i/p elements in matrix

n^3 = for finding shortest path

$$\therefore n^2 + n^3 = \underline{\underline{O(n^3)}}$$

↳ $n = \text{num (vertices)}$

Algo (A, cost)

```
{
  for i=1 to n do
  for j=1 to n do
    A[i,j] = cost[i,j];
  } n^2
```

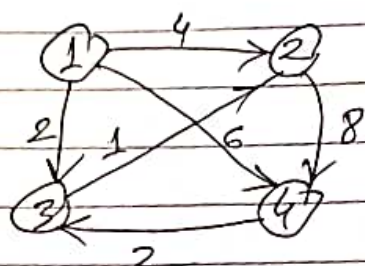
```
  for k=1 to n do
  for i=1 to n do
  for j=1 to n do
  } n^3
```

$A[i,j] = \min [A[i,j], A[i,k] + A[k,j]],$

}

Every new Table (matrix) is Base for next Table (matrix)

Ex →



solve using Floyd Warshall

Soln → Initial = $D_0 =$

	1	2	3	4
1	0	4	2	6
2	∞	0	∞	8
3	∞	1	0	∞
4	∞	∞	2	0

1st

①

$$D_1 =$$

	1	2	3	4
1	0	4	2	6
2	∞	0	∞	8
3	∞	1	0	∞
4	∞	∞	2	0

2nd

②

$$D_2 =$$

	1	2	3	4
1	0	4	2	6
2	∞	0	∞	8
3	∞	1	0	9
4	∞	∞	2	0

3rd

③

$$D_3 =$$

	1	2	3	4
1	0	3	2	6
2	∞	0	∞	8
3	∞	1	0	9
4	∞	3	2	0

4th

④

$$D_4 =$$

	1	2	3	4
1	0	3	2	6
2	∞	0	10	8
3	∞	1	0	9
4	∞	3	2	0

$D_4 =$ shortest path of all vertices

"Travelling salesman person problem"

→ dynamic programming approach

formula, $T(i, s) = \begin{cases} \min_{j \in s} ((i, j) + T(j, s - j)) & i \rightarrow s \neq \emptyset \\ (i, 1) & s = \emptyset \end{cases}$

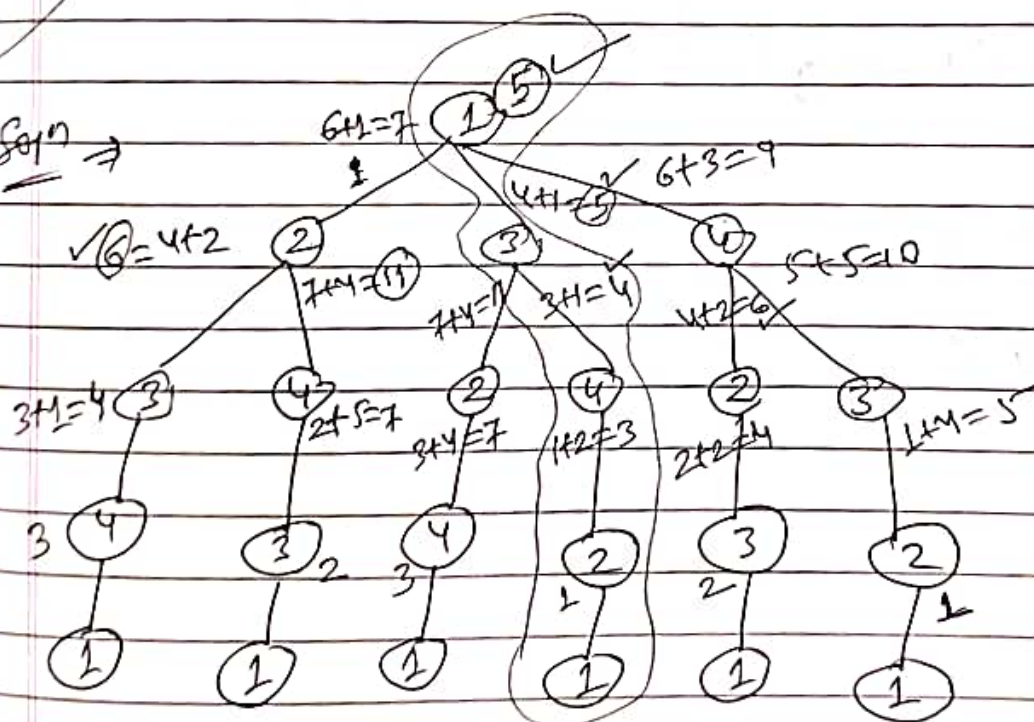
starting vertex set of vertex

Ex →

	1	2	3	4
1	0	1	2	3
2	1	0	4	2
3	1	2	0	5
4	3	4	1	0

Find min cost using TSP?

Soln →



shortest path = 1 2 4 3 1

cost = 5

Note (TSP)

① Time Complexity = $O(n^2 2^n)$

② Space Complexity = $O(n 2^n)$

~~How many~~ $\text{num}(\text{Subproblem}) = (n-1) \times \frac{n-2}{2}$

③ $\text{num}(\text{Subproblem}) = (n-1) \times 2^{n-2}$

Ex $\Rightarrow$ for question $(1, 2, 3, 4)$ ~~max~~
 $\text{num}(\text{Subproblem}) = \downarrow \text{num}(\text{elements})$
 $(4-1) \times 2^{4-2}$

$= 3 \times 2^2 \Rightarrow 12$

"Reliability design"

Devices D_i

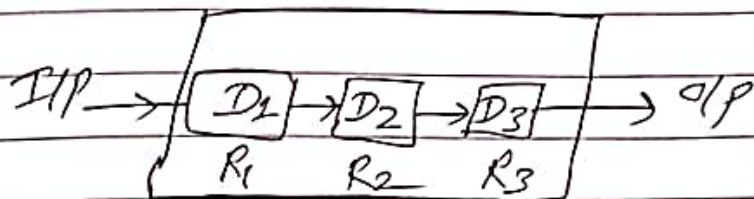
Reliability R_i

Cost C_i

$D_1 \quad D_2 \quad D_3 \quad \dots$

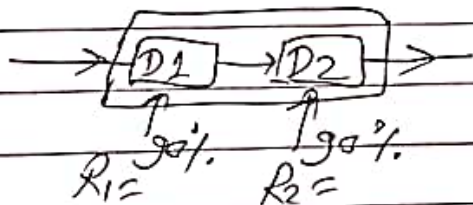
$R_1 \quad R_2 \quad R_3 \quad \dots$

$C_1 \quad C_2 \quad C_3 \quad \dots$
 $\underbrace{\hspace{10em}}_{\text{Cost}}$



$$\therefore \text{Reliability} = R_1 \times R_2 \times R_3$$

Ex $\Rightarrow$



$$\therefore \text{System Reliability} = R_1 \times R_2 = \frac{90}{100} \times \frac{90}{100}$$

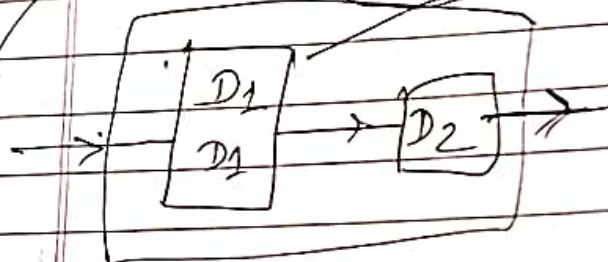
$$= \frac{81}{100} = 0.81 \times 100$$

$$\textcircled{81} \quad 81 \times \frac{1}{100} \Rightarrow \underline{\underline{81\%}}$$

$$\textcircled{\% = \frac{81}{100}}$$

Improve Reliability

→ 99.9%



m = num of device repeated

$$R_{s1} = \left[1 - (1 - z_1)^m \right]$$
$$= \left[1 - (1 - 0.9)^2 \right]$$

z = ^{existing} reliability

$$= 1 - 0.01$$

$$\underline{\underline{= 99.9\%}}$$



D_1	D_2
C_1	C_2
R_1	R_2

Cost of system = C

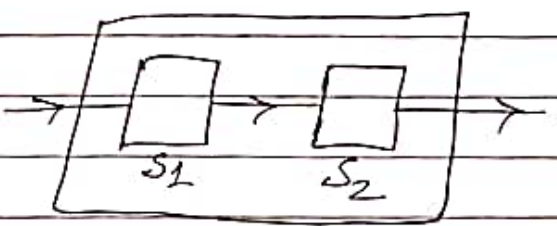
$$\text{num (devices) of a stage} \Rightarrow \left\lfloor \frac{C + C_1 - \sum_{i=1}^n C_i}{C_1} \right\rfloor$$

$\leftarrow$ floor from $(7.15) = 7$

Ex $\Rightarrow C = 100, D_1 = 90\%, D_2 = 70\%$
 $C_1 = 30, C_2 = 20$

num of copy for $D_1 = \left\lfloor \frac{(100 + 30 - 50)}{30} \right\rfloor$

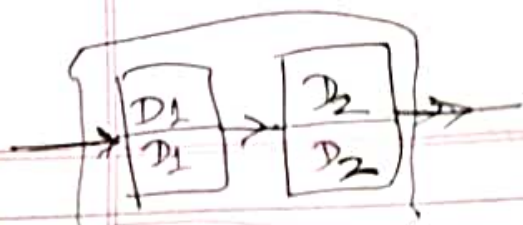
$= \left\lfloor 80/30 \right\rfloor = 2$



num of copy for $D_2 = \left\lfloor \frac{(100 + 20 - 50)}{20} \right\rfloor$

$= \left\lfloor 70/20 \right\rfloor = 3$

$D_1 = 2 \text{ copy}$
 $(1 - (1 - 0.9)^2)$
 ≈ 0.99



$$S^0 = \left\{ \binom{R}{(1, 0)} \right\}$$

Stage 1 $S_1^1 = \{ (0.9, 30) \}$

$$S_2^1 = \{ (0.99, 60) \}$$

$$S^1 = \{ (0.9, 30), (0.99, 60) \}$$

Stage 2

$$S_2^1 \Rightarrow \{ (0.63, 20), (0.693, 80) \}$$

$$S_2^2 \Rightarrow \{ (0.819, 70), (0.9, 120) \}$$

$$R_1 = (1 - (1 - 0.7)^2)$$

$$R_1 = (1 - (1 - 0.7)^3)$$

$$S_2^3 \Rightarrow \{ (0.875, 90), (, 120) \}$$

max
limit
reached

$$S_2^2 \Rightarrow (0.9, 100)$$

90% Reliable with

cost 100 & 2 devices.